Implementation of Peer-to-Peer Energy Trading via Solana Smart Contracts in a Permissioned Environment

Chanthawat Kiriyadee[1], and Suwannee Adsavakulchai[1*]

[1]Department of Computer Engineering and Artificial Intelligence, School of Engineering, University of the Thai Chamber of Commerce (UTCC), Bangkok, Thailand

Contact Email: *2410717302003@live4.utcc.ac.th, Suwannee_ads@utcc.ac.th*

Thailand's three state utilities — EGAT, MEA, and PEA — have pursued a National Energy Trading Platform (NETP) since 2017 for peer-to-peer (P2P) trading among rooftop-solar prosumers, but it remains in development, and the regulated buy-back affords prosumers no direct means of exchanging surplus within the local distribution grid. This paper evaluates a settlement layer for that role: six modular Anchor programs on a permissioned Solana runtime in which every high-frequency write is routed to a per-entity program-derived account (PDA), so transactions of distinct entities hold disjoint write sets and are scheduled in parallel by the Sealevel engine, ordered by Proof-of-History [1]. We measure compute, throughput and latency together, and find that they are not bound by the same thing. Compute is $O(1)$ and never binds: the steady telemetry write holds 13.3–13.6 k compute units (CU) across a 2,500× sweep from 80 to 200,000 meters and against 510,000 resident PDAs, while confirmation latency stays flat at p50 ≈ 1.5–1.9 s (p95 ≤ 3.1 s). Throughput is instead set by **lock footprint** rather than instruction cost. Ordering every measured path by the shared write-locked state it touches reproduces a three-order-of-magnitude spread, and a controlled before/after isolates the mechanism: while the sharded settle path declared its shared parent account writable, same-zone settles could not co-execute and landed at one per slot; making that account read-only — the handler never wrote it — packs 3.00 settles per slot, a 2.7–3× gain from deleting a single `mut`.

1. Introduction

Since 2017 Thailand's state utilities have pursued a National Energy Trading Platform (NETP) for P2P trading among rooftop-solar prosumers, implemented on a Tendermint-based consortium blockchain [2]. This article presents a Solana-based architecture as an alternative execution layer, motivated by NETP's own evaluation, in which Tendermint outperformed both Ethereum and Hyperledger Fabric under throughput testing: that result suggests further gains from a runtime whose execution is parallel rather than sequential. Such evaluations report throughput and compute cost as though computation were the axis along which a settlement layer scales. The contribution here is to separate the axes — to partition all hot-path state per entity, then measure compute, throughput and latency independently, so that what binds each can be named rather than attributed to computation by default.

2. The Per-Entity Partition

The settlement layer is six Anchor programs [3] (anchor-lang and anchor-spl 1.0.0) — **energy-token, governance, oracle, registry, trading,** and **treasury** — in Rust, compiled to SBF bytecode. Programs invoke one another through cross-program invocation (CPI), authority delegated to PDAs that sign without any private key. A PDA is the SHA-256 of its seed tuple, a bump byte, the program id and a fixed domain-separation constant, accepted only if the 32 bytes are **not** a valid Ed25519 curve point, so no private key can ever exist for it and the runtime may safely let the owning program sign as that address. A transaction declares in advance every account it touches and whether each is writable; the runtime locks per account before execution, so two transactions with disjoint writable sets run on different cores and two writing one account serialise. Table 1 gives the hot-path partition under that rule.

Table 1: **Hot-path state partition.** Every high-frequency write targets its own program-derived account, so any two are Sealevel-parallelisable.

Hot path	Account	Seed tuple
meter telemetry	`MeterState`	`[b"meter", meter_id]` — the 32-byte id ceiling is the runtime's own seed limit
order placement	`Order`	`[b"order", authority, order_id]`
settlement replay guard	`OrderNullifier`	`[b"nullifier", user, order_id]`
registration counters	`RegistryShard` ×16	`[b"registry_shard", shard_id]`, shard $=$ key$[0] \bmod 16$

Two consequences follow. First, global accounts — config, totals — are read-only on hot paths and **deliberately stale**; periodic admin instructions fold per-entity state back into them off the hot path. Second, derivation is itself a compute budget item: `find_program_address` searches the bump byte downward from 255, paying a hash plus a curve-decompression check per candidate and terminating only probabilistically. The programs therefore store the winning bump in the account and thereafter validate with a single `create_program_address` — **1,651 CU against the 12,136 CU** a fresh search costs on the aggregation path, a 7.3× saving on pure addressing.

3. Method

All experiments ran on one node (Apple M2, Agave 3.1.10) of a private `solana-test-validator` network, so they characterise SVM semantics — account locking and compute metering — not consensus or leader rotation [4]. Throughput is **client-observed confirmed goodput**: confirmed transactions per wall-clock second from burst start to last confirmation, covering the full pipeline from client signing to confirmation visibility. Every non-confirmed transaction is attributed to one class — send-rejected (refused by the RPC node, no fee), guard-rejected (executed, failed a program check, fee paid and recorded), or expired — separating delivery loss from validation working as intended. Transactions are pre-signed against a cached blockhash and submitted raw without preflight or retry, then confirmed by bulk signature-status polling on a 1.5 s sweep under a 3,000-transaction in-flight window; latency is therefore poll-quantised (±1.5 s) and bounds the runtime's own cost from above rather than measuring it. CU is machine-independent, read post-hoc from transaction metadata. The telemetry sweep submits one reading per meter from N distinct meters over two epochs at least 61 s apart — the on-chain rate-limit and strictly-increasing-timestamp guards force the gap — which separates one-off PDA creation from the recurring steady write. It covers $N = 80$ to 200,000 (1.35 M transactions); later scales run unreset, so the largest writes against 310,000 already-resident PDAs and the sweep ends holding 510,000. The order-entry burst submits $N = 10^4$ orders over 16 round-robin zone shards on the same transport and the same fee payer, which makes the two directly comparable.

4. Results and Discussion

Compute is $O(1)$ and never binds. Table 2 gives the instruction profile against the 200,000-CU default per-instruction budget. Every control, telemetry and settlement-recording path costs at most ≈8% of it. The steady telemetry write holds 13,337–13,634 CU across a 1,250-fold rise in meter count, with the residual ±150 CU attributable to meter-identifier byte length rather than fleet size, and the first write for a meter costs 16,050 CU because it initialises and rent-funds the PDA. Under a concurrency sweep from 5 to 40 in-flight transactions cost is likewise flat at 22–24 k CU, so execution is load-independent as well as fleet-independent. Settlement is the one expensive instruction at 121,813 CU (60.9% of budget), and its cost is dominated by native Ed25519 verification and instruction-sysvar introspection authorising the trade — not by settlement arithmetic.

Table 2: **Compute cost per instruction**, against the 200,000-CU default per-instruction budget. Machine-independent, read from transaction metadata.

Instruction	CU	% of 200 k
`do_nothing` (dispatch + signature-verify floor)	769	0.4%
`treasury.record_settlement`	3,301	1.7%
`treasury.record_settlement_sharded`	5,374	2.7%
`trading.submit_limit_order_sharded`	9,884	4.9%
`oracle.submit_meter_reading` (steady)	13,376	6.7%
`oracle.submit_meter_reading` (first, inits PDA)	16,050	8.0%
`trading.settle_offchain_match`	121,813	60.9%

Compute is also $O(1)$ in accumulated state: the last 200,000 submissions execute against a state set five times larger than the first scale's, indistinguishable in cost. The mechanism is the account store — payloads live in append-only files, an update appends rather than rewriting in place, and lookup goes through an in-memory pubkey → (slot, offset) index, so a write over 510,000 accounts costs one probe and one append exactly as over 10,000. What scales with account count is index memory, snapshot size and startup time: multi-node operational concerns, invisible to per-transaction cost.

Throughput is bound by lock footprint, not by instruction cost. Table 3 sorts every measured path by the shared writable state it touches, and the ordering — not the CU column — predicts the rate across three orders of magnitude. The mechanism is isolated by a controlled before/after on one path, holding the harness fixed and changing one account attribute. While the sharded settle context declared the parent `ZoneMarket` writable, same-zone settles shared that lock and landed at no more than one per slot — they cannot co-execute by construction; with the account made read-only, which the handler's write set always permitted, the validator packs 3.00 settles·slot^{-1} at $N = 40$. A 2.7–3× gain follows from deleting one `mut` and changing nothing else. The same defect accounts for order entry: measured at $N = 10^4$ on the same transport and fee payer as telemetry, it returned 180 TPS against 462 while being the **cheaper** instruction (9,884 CU against 13,337), because its context then declared the shared zone-market account writable although the handler mutated only the per-shard account, serialising all 16 shards behind one lock. That `mut` has since been dropped; the path has not been re-measured. Telemetry, holding only the gateway fee payer, is flat at 426–490 TPS from 5,000 to 200,000 meters — not proportional, since one payer signs everything, but not degrading either, since the per-meter PDAs never contend.

Table 3: **Throughput by lock footprint.** Every measured path, ordered by the shared write-locked state it touches. Harnesses differ per row; rows are not mutually comparable. *pre-fix* = measured before the vestigial `mut` on `zone_market` was dropped; not since re-measured.

Path	CU	TPS	Shared writable state
RPC read	—	≈1,454	none — no consensus round-trip
meter ingest (5 k–200 k meters)	13.3–13.6 k	426–490	gateway fee payer
order entry (10 k orders, pre-fix)	9,884	180	fee payer + `zone_market`
OLTP proxy (concurrency 5 → 40)	22–24 k	7.9 → 74.3	fee payer + market accounts
settlement, single fee payer	≈115 k	≈0.6	fee payer + collectors + `treasury_state`

Latency is set by block time, not by the partition. Confirmation holds p50 ≈ 1.5–1.9 s and p95 ≤ 3.1 s across the whole 2,500× fleet range — flat where a contended design would degrade — and order entry sits at p50 2,173 ms, p95 3,565 ms under its extra lock. Both are send→first-seen-confirmed under the 1.5 s poll: upper bounds dominated by the ≈400–600 ms single-node block time, not measurements of runtime cost. The scale of that domination is visible in the read path, which needs no consensus round-trip and returns in under a millisecond — three orders of magnitude faster than any write.

Delivery loss is zero across the 1.02 M first-attempt sends of the unreset sweep; the 0.46–0.48% residue is a deterministic anomaly-gate probe firing on the same meter indices every epoch, and because the rate-limit and monotonic-timestamp guards make resubmission idempotent, one retry round would put true loss on the order of 10^{-5}.

Together these establish the central claim: with hot-path state partitioned per entity, neither computation nor per-entity contention bounds the system — what remains is serialisation on the few genuinely shared accounts, and a single validator bounds execution and locking only, not consensus in a three-utility consortium.

5. Conclusion

Per-entity PDA partitioning removes computation as the scaling limit of a P2P energy settlement layer: the steady write is $O(1)$ in fleet size and in resident account count, and latency is flat over a 2,500× sweep. What binds instead is lock footprint, and it stands in inverse relation to instruction cost: the cheaper order-entry write ran 2.6× slower than telemetry for one extra shared writable account. The lever is therefore to shrink declared write sets, not to optimise compute — narrowing account contexts to what a handler actually mutates is worth 2.7–3× on the settlement path for a one-line change, and sharding global accumulators and pooling fee payers remain untested above it. Reproducing these results on a multi-validator consortium mapped to EGAT, MEA, and PEA would place consensus itself under test, and suits UEC–ASEAN collaboration.

Acknowledgment

The authors thank the University of the Thai Chamber of Commerce (UTCC) for institutional support, and the organizers, the UEC ASEAN Research Center (UARC) and Multimedia University (MMU), for hosting this workshop.

References

[1] A. Yakovenko, "Solana: A New Architecture for a High Performance Blockchain v0.8.13." 2018.
[2] Metropolitan Electricity Authority (MEA) and Electricity Generating Authority of Thailand (EGAT) and Provincial Electricity Authority (PEA), "Development of the National Energy Trading Platform (NETP): Research and Development Project Report," technical report, Dec. 2019.
[3] Coral / Anchor contributors, "Anchor: Solana's Sealevel runtime framework." [Online]. Available: https://www.anchor-lang.com/
[4] Solana Foundation, "Tower BFT: Solana's High Performance Implementation of PBFT." Jul. 2019.